Reviewer Pack — Width-5 ABPs and NC¹ Circuit Simulation

Entry cd771dc62fac · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 04:23:09 UTC

Width-5 ABPs and NC¹ Circuit Simulation

Entry ID: `cd771dc62fac` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-29 04:23:09 UTC

1. Conjecture

Field A (mathematical branch): Barrington's theorem **Field B** (complexity object): Algebraic branching programs

Statement:

> Any function computable by a depth-d NC¹ circuit can be simulated by a width-5 algebraic branching program (ABP) of size O(n^d), but there exist functions in P that require ABPs of exponential size to compute.

Rationale (proposer's reasoning):

> Barrington's theorem establishes that width-5 ABPs can simulate NC¹ circuits, but the size of the ABP depends on the depth of the circuit. This conjecture extends the separation by proposing a polynomial-size ABP for NC¹ functions while suggesting exponential lower bounds for P functions, aligning with Barrington's techniques and avoiding barriers like relativization.

Taxonomy category: `BARRINGTON_ALG` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `89c4e6d114e7f403`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def matrix_multiply(A, B):
        n = len(A)
        C = [[0 for _ in range(n)] for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def gaussian_elimination(A, b):
        n = len(A)
        for i in range(n):
            max_row = i
            for j in range(i+1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            b[i], b[max_row] = b[max_row], b[i]
            for j in range(i+1, n):
                factor = A[j][i] / A[i][i]
                for k in range(i, n):
                    A[j][k] -= factor * A[i][k]
                b[j] -= factor * b[i]
        x = [0 for _ in range(n)]
        for i in range(n-1, -1, -1):
            x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
        return x

    def generate_nc1_circuit(depth: int):
        if depth == 0:
            return random.choice([0, 1])
        else:
            a = generate_nc1_circuit(depth-1)
            b = generate_nc1_circuit(depth-1)
```

```
        return (a + b) % 2

def simulate_abp(circuit, abp):
    n = len(abp)
    state = [0] * n
    for node in circuit:
        if isinstance(node, int):
            state[node] += 1
        else:
            a = state.pop()
            b = state.pop()
            state.append((a + b) % 2)
    return state[0]

def abp_size(circuit):
    n = len(circuit)
    size = 0
    for node in circuit:
        if isinstance(node, int):
            size += 1
        else:
            size += 2
    return size

def generate_p_function(n):
    return [random.choice([0, 1]) for _ in range(2**n)]

def simulate_abp_for_p(abp, p_function):
    n = len(p_function)
    state = [0] * (2**n)
    for i in range(2**n):
        for j in range(n):
            if (i >> j) & 1:
                state[i] += p_function[j]
        state[i] %= 2
    return state

def abp_size_for_p(abp, p_function):
    n = len(p_function)
    size = 0
    for i in range(2**n):
        for j in range(n):
            if (i >> j) & 1:
                size += 1
        size += 2
    return size

def generate_nc1_circuit_of_depth(depth: int):
    circuit = []
    for _ in range(depth):
        if random.choice([0, 1]) == 0:
            circuit.append(random.randint(0, len(circuit)-1))
        else:
            circuit.extend([len(circuit), len(circuit)+1])
    return circuit
```

```
def generate_abp_for_nc1_circuit(circuit):
    n = len(circuit)
    abp = [0] * (2*n)
    for i in range(n):
        if isinstance(circuit[i], int):
            abp[circuit[i]] += 1
        else:
            a = circuit[i]
            b = circuit[i+1]
            abp[a] += 1
            abp[b] += 1
            abp[2*n-1] += 1
    return abp

def generate_abp_for_p_function(p_function):
    n = len(p_function)
    abp = [0] * (2**n + 2*n)
    for i in range(2**n):
        for j in range(n):
            if (i >> j) & 1:
                abp[i] += p_function[j]
        abp[i] %= 2
    return abp

def generate_counterexample():
    n = random.randint(5, 30)
    p_function = generate_p_function(n)
    abp = generate_abp_for_p_function(p_function)
    if abp_size_for_p(abp, p_function) > 2**n:
        return f"Exponential size ABP for P function of length {n}"
    else:
        return ""

def run_nc1_circuit_simulation(depth: int):
    circuit = generate_nc1_circuit_of_depth(depth)
    abp = generate_abp_for_nc1_circuit(circuit)
    return abp_size(circuit), simulate_abp(circuit, abp)

def run_p_function_simulation():
    n = random.randint(5, 30)
    p_function = generate_p_function(n)
    abp = generate_abp_for_p_function(p_function)
    return abp_size_for_p(abp, p_function), simulate_abp_for_p(abp, p_function)

results = []
for _ in range(30):
    depth = random.choice([5, 10, 15, 20, 30, 40])
    nc1_circuit_size, nc1_circuit_result = run_nc1_circuit_simulation(depth)
    p_function_size, p_function_result = run_p_function_simulation()
    results.append({
        "metric_name": "ABP Size",
        "metric_value": nc1_circuit_size,
        "instances_tested": 1,
        "conjecture_holds": nc1_circuit_size <= depth**2 * n,
        "counterexample": generate_counterexample()
    })
```

```
    return {
        "metric_name": "Average ABP Size",
        "metric_value": sum(r["metric_value"] for r in results) / len(results),
        "instances_tested": 30,
        "conjecture_holds": all(r["conjecture_holds"] for r in results),
        "counterexample": next((r["counterexample"] for r in results if r["counterexample"]), "")
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction:.2f
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = next((r["counterexample"] for r in results if r["counterexample"]), "")
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8438b265.py", line 181, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8438b265.py", line 159, in run_trial
    p_function_size, p_function_result = run_p_function_simulation()
                                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8438b265.py", line 152, in run_p_functi
    abp = generate_abp_for_p_function(p_function)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8438b265.py", line 127, in generate_abp
    abp = [0] * (2**n + 2*n)
          ~~~~^~~~~~~~~~~~~~
OverflowError: cannot fit 'int' into an index-sized integer
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

Test crashed due to overflow error, preventing evaluation of conjecture | next: Optimize ABP generation algorithm to handle large n values

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	107108
2	preregistration	ollama_remote	qwen3:8b	0	0	24136
3	novelty	ollama_remote	qwen3:8b	0	0	22839
4	novelty	ollama_remote	qwen3:8b	0	0	20864
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17025
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26378
7	judge	ollama_remote	qwen3:8b	0	0	14666

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 233017 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/cd771dc62fac.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cd771dc62fac.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/cd771dc62fac.tar.gz` (if generated)